

```

// Berkeley Clock synchronization algorithm

#include<stdio.h>
#include<string.h>
#include<mpi.h>

int main()
{
    int rank, size, i;

    float indv_time, ts_time, corr_time, sum, avg;
    char msg[20];

    MPI_Status s;

    MPI_Init(NULL, NULL);

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if(rank == 0)
    {
        strcpy(msg, "Send your time");
        MPI_Bcast(msg, sizeof(msg), MPI_CHAR, 0, MPI_COMM_WORLD);

        printf("\n Enter time for time server:");
        scanf("%f", &ts_time);

        sum=ts_time;
        for(i=1;i<=(size-1);i++)
        {
            MPI_Recv(&indv_time, 1, MPI_FLOAT, MPI_ANY_SOURCE, 111, MPI_COMM_WORLD, &s);
            sum = sum + indv_time;
        }

        avg = sum/size;
        corr_time = avg;

        MPI_Bcast(&avg, 1, MPI_FLOAT, 0, MPI_COMM_WORLD);
    }

    if(rank !=0 )
    {
        MPI_Bcast(msg, sizeof(msg), MPI_CHAR, 0, MPI_COMM_WORLD);
        if(strcmp(msg, "Send your time") == 0)
        {
            printf("\n Enter time for process%d:", rank);
            scanf("%f", &indv_time);
        }
    }
}

```

```

    MPI_Send(&indv_time, 1, MPI_FLOAT, 0, 111, MPI_COMM_WORLD);
    MPI_Bcast(&corr_time, 1, MPI_FLOAT, 0, MPI_COMM_WORLD);
}
}

MPI_Barrier(MPI_COMM_WORLD);

printf("\n Corrected time for process%d = %f",rank,corr_time);

MPI_Finalize();
return 0;
}

```

Output

```

mpicc clk_syn.c -o clk
mpirun -np 5 clk

```

```

Enter time for time server:
Enter time for process1:
Enter time for process2: Enter time for process3:
Enter time for process4:300
290
280
325
310

```

```

Corrected time for process0 = 301.000000
Corrected time for process1 = 301.000000
Corrected time for process2 = 301.000000
Corrected time for process3 = 301.000000
Corrected time for process4 = 301.000000

```